

# Volume 15

## Enterprise API Architecture (EAA)

**Version:** 1.0

**Classification:** Enterprise Architecture

**Status:** Draft

## Table of Contents

**15.1 Purpose**

**15.2 API Vision**

**15.3 API Principles**

**15.4 API Architecture Overview**

**15.5 API Classification**

**15.6 REST API Standards**

**15.7 API Versioning Strategy**

**15.8 API Security**

**15.9 API Gateway Architecture**

**15.10 Request Processing Pipeline**

**15.11 API Documentation**

**15.12 API Lifecycle Management**

**15.13 Error Handling Standards**

**15.14 Rate Limiting & Throttling**

## 15.15 Webhooks

## 15.16 WebSockets & Real-Time Communication

## 15.17 Future API Evolution

## 15.18 Volume Summary

# 15.1 Purpose

The Enterprise API Architecture defines the standards, principles, and governance model for all application programming interfaces exposed by the BiteFixes platform.

The objectives of this architecture are to:

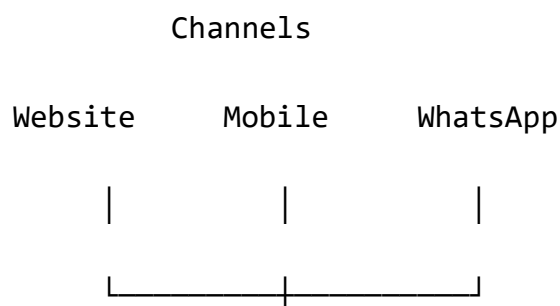
- Ensure consistency across all APIs.
- Support secure communication.
- Enable interoperability between services.
- Simplify integration with external systems.
- Maintain backward compatibility.
- Support long-term scalability.

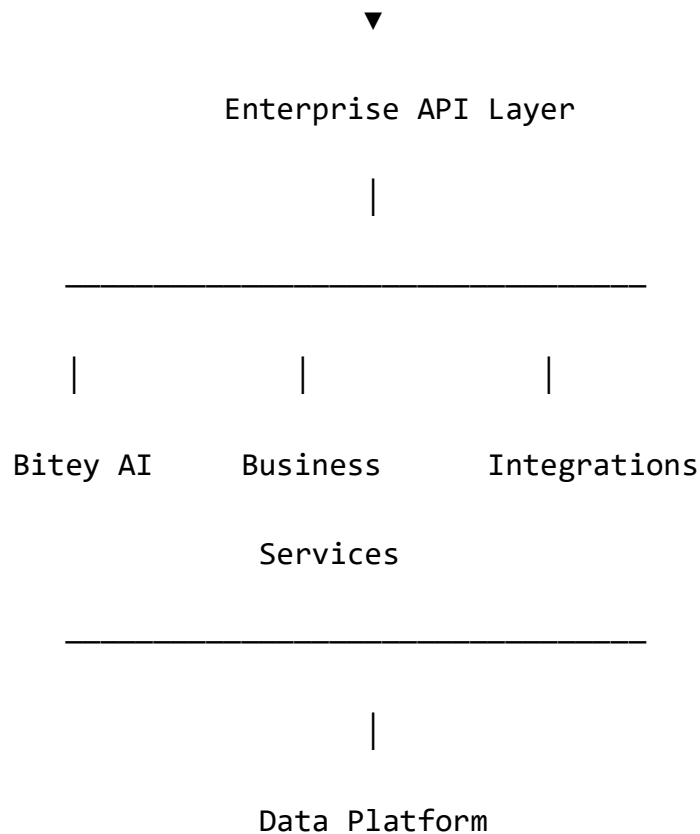
Every public, private, and internal API must comply with this architecture.

# 15.2 API Vision

APIs are the communication backbone of the BiteFixes platform.

Every business capability is exposed through well-defined service interfaces rather than direct access to internal components.





This architecture ensures that every client consumes the same business capabilities through standardized interfaces.

## 15.3 API Principles

### Principle 1 — API First

Business functionality is designed as reusable APIs before user interfaces are implemented.

Benefits include:

- Consistency
- Reusability
- Faster development
- Easier testing
- Better integrations

## Principle 2 — Stateless Communication

Each request contains all information required for processing.

No server-side session state is assumed for API requests.

Authentication and authorization context are supplied with each request.

## Principle 3 — Resource-Oriented Design

APIs expose business resources rather than implementation details.

Examples:

Customers

Devices

Tickets

Services

Companies

Knowledge

Conversations

Users

Endpoints represent resources and standard HTTP methods define the operations.

## Principle 4 — Consistent Contracts

Every endpoint follows common conventions for:

- Naming
- HTTP methods

- Status codes
- Error responses
- Pagination
- Filtering
- Authentication

Consistency reduces integration complexity.

## Principle 5 — Backward Compatibility

Breaking changes are avoided whenever possible.

When incompatible modifications are necessary, a new API version is introduced rather than altering existing behavior.

## 15.4 API Architecture Overview

The Enterprise API Layer sits between external consumers and internal business services.

External Client

|



API Gateway

|



Authentication

|



Authorization

|



Business Service

|



Repository Layer

|



Database

Each request passes through the complete processing pipeline before reaching business logic.

## 15.5 API Classification

The platform distinguishes several API categories.

### Public APIs

Accessible without authentication.

Examples:

- Health checks
- Public service catalog
- Public company information

## Authenticated Customer APIs

Accessible only to authenticated customers.

Examples:

- View tickets
- Create support requests
- Update profile
- View repair status

## Technician APIs

Used by repair personnel.

Examples:

- Update diagnostics
- Modify ticket status
- Register repairs
- Add technical notes

## Administrative APIs

Reserved for company administrators.

Examples:

- Manage users
- Configure services
- Manage pricing
- Configure business rules

## Platform APIs

Reserved for BiteFixes platform administrators.

Examples:

- Tenant management
- Platform monitoring
- Subscription management
- Global configuration

## Internal APIs

Never exposed publicly.

Used exclusively for communication between internal services.

Examples:

Identity Service

Notification Service

AI Service

Audit Service

Knowledge Service

## Integration APIs

Used by third-party systems.

Examples:

- WhatsApp
- WordPress
- ERP
- CRM
- Payment providers
- Mobile applications



## 15.6 REST API Standards

### Resource Naming

Resources use plural nouns.

Examples:

GET /customers

GET /devices

GET /tickets

GET /services

GET /companies

### HTTP Methods

The platform follows standard REST semantics.

| Method | Purpose            |
|--------|--------------------|
| GET    | Retrieve resources |
| POST   | Create resources   |
| PUT    | Replace resources  |
| PATCH  | Partial update     |
| DELETE | Remove resources   |

### URI Design

Hierarchical relationships are represented through nested resources where appropriate.

Examples:

/customers/{id}

/customers/{id}/devices

/customers/{id}/tickets

/tickets/{id}/comments

/companies/{id}/users

## Response Format

All responses use a consistent JSON structure.

Successful responses include:

- Resource data
- Metadata (when applicable)
- Pagination information (for collections)

Error responses follow a standardized schema defined later in this volume.

## HTTP Status Codes

Standard status codes are used consistently:

| Code | Meaning               |
|------|-----------------------|
| 200  | Success               |
| 201  | Resource Created      |
| 204  | No Content            |
| 400  | Bad Request           |
| 401  | Unauthorized          |
| 403  | Forbidden             |
| 404  | Not Found             |
| 409  | Conflict              |
| 422  | Validation Error      |
| 429  | Too Many Requests     |
| 500  | Internal Server Error |

## Idempotency

Operations that modify state should support idempotency where appropriate.

Examples include:

- Payment confirmations
- Webhook processing
- Retryable API operations

Unique request identifiers may be used to detect duplicate submissions.

## Chapter Summary

The first half of the Enterprise API Architecture establishes the strategic vision, guiding principles, architectural layers, API categories, and REST standards that govern every interface exposed by the BiteFixes platform. By adopting an API-first approach with consistent contracts, standardized resource design, and clear separation of concerns, the platform ensures maintainability, interoperability, and long-term scalability.